

GuardianClima ITBA

Documento del Proyecto - Challenge Tecnológico Integrador

Equipo	Equipo Numero 1
Integrantes	Lucas Garbate, Tomas Grinstein, Bautista Cavalitto, Facundo Gatti
Materia	Tecnología ITBA
Fecha	Mayo 2026

1. Introduccion

GuardianClima ITBA es una aplicacion de consola desarrollada en Python como parte del Challenge Tecnologico Integrador. Permite consultar el clima actual de cualquier ciudad, guardar un historial global de consultas, visualizar estadisticas de uso y recibir consejos de vestimenta generados por inteligencia artificial.

1.1 Objetivos

- Desarrollar una aplicacion funcional de consola en Python.
- Integrar conceptos de Programacion, Ciberseguridad, Analisis de Datos, Inteligencia Artificial y Cloud Computing.
- Implementar validacion de contraseñas con criterios de seguridad reales.
- Conectar la aplicacion a APIs externas de clima e IA.
- Gestionar datos persistentes mediante archivos CSV.

1.2 Tecnologias utilizadas

Componente	Tecnologia / Servicio
Lenguaje	Python 3.x
API de Clima	wtrr.in (gratuita, sin API key)
API de IA	Google Gemini (gemini-flash-latest via REST)
Persistencia	Archivos CSV (users.csv, historial_global.csv)
HTTP	Libreria requests
Variables de entorno	python-dotenv

2. Diseño y Arquitectura

2.1 Estructura de archivos

```
src/main.py - Punto de entrada. Maneja los dos menus y el estado de sesion.  
src/service.py - Logica de negocio: registro, login, clima, historial,  
estadisticas, IA.  
src/lib/storage.py - Lectura y escritura de los archivos CSV.  
src/lib/ia.py - Integracion con Google Gemini via HTTP REST.  
src/lib/clima.py - Script auxiliar de prueba para la API de clima.  
db/users.csv - Almacena los usuarios registrados.  
db/historial_global.csv - Historial de todas las consultas de clima.
```

2.2 Flujo de la aplicacion

- **Pre-login (Menu de Acceso):** el usuario puede registrarse o iniciar sesion. Las credenciales se validan contra users.csv. Tras registro o login exitoso la sesion queda activa y pasa al Menu Principal.
- **Post-login (Menu Principal):** el usuario accede a las 6 opciones. Las consultas se guardan en historial_global.csv. Las estadisticas y el historial personal se calculan leyendo ese archivo en tiempo real.

2.3 Decisiones de diseno

- **wtrr.in en lugar de OpenWeatherMap:** no requiere API key ni registro, simplificando la configuracion.
- **Gemini via HTTP directo:** se usa requests en lugar de google-generativeai para mayor control del endpoint.
- **Modularidad:** logica separada en service.py (negocio) y lib/ (integraciones), dejando main.py solo para navegacion.
- **.env para las API keys:** las claves se cargan con python-dotenv desde un archivo excluido del repositorio.

3. Guia de Usuario Detallada

3.1 Menu de Acceso

Al iniciar la aplicacion se presentan tres opciones:

- **1) Crear usuario**
 - Solicita nombre de usuario y contrasena.
 - La contrasena debe cumplir los 5 criterios de seguridad (ver abajo).
 - Si no los cumple, informa que reglas fallaron y sugiere como mejorarla.
 - Tras el registro exitoso, inicia sesion automaticamente.
- **2) Iniciar sesion**
 - Solicita nombre y contrasena.
 - Si son correctas, accede al Menu Principal.
 - Si no, muestra error y permite reintentar.
- **3) Salir**
 - Termina la ejecucion del programa.

3.2 Criterios de seguridad de contrasena

La contrasena debe cumplir **todos** los siguientes criterios:

- Minimo 8 caracteres.
- Al menos una letra mayuscula.
- Al menos una letra minuscula.
- Al menos un numero.
- Al menos un caracter especial (!@#\$%^&*(),.?"':{}|<>).

Ejemplo valido: **MiPerro#Tiene7Annos!**

3.3 Menu Principal

Una vez autenticado, el usuario accede a seis opciones:

- **1) Consultar clima**
 - Ingresa el nombre de una ciudad.
 - Muestra temperatura, sensacion termica, humedad, viento y condicion.
 - La consulta se guarda automaticamente en `historial_global.csv`.
- **2) Ver historial personal**
 - Ingresa una ciudad.

- Muestra todas tus consultas previas para esa ciudad ordenadas por fecha.

- **3) Estadísticas globales**

- Muestra la ciudad mas consultada entre todos los usuarios.

- Muestra el total de consultas y la temperatura promedio global.

- **4) Consejo IA**

- Usa los datos de tu ultima consulta de clima.

- Consulta a Google Gemini y muestra un consejo de vestimenta personalizado.

- **5) Acerca de**

- Muestra descripcion de la app, guia de uso, funcionamiento interno y datos del equipo.

- **6) Cerrar sesion**

- Finaliza la sesion actual y vuelve al Menu de Acceso.

4. Desafios y Soluciones

API de clima — OpenWeatherMap no disponible

Al intentar usar OpenWeatherMap la activacion de la API key tardaba demasiado. Se reemplazo por wttr.in, un servicio gratuito y sin registro que devuelve los mismos datos en formato JSON.

Cuota agotada en Google Gemini (limite 0)

La primera API key de Gemini tenia limite 0 en el free tier por restricciones regionales. Se genero una nueva key desde Google AI Studio y se identifico el modelo correcto consultando directamente el endpoint /v1beta/models.

Error 404 al llamar a la API de Gemini

El nombre de modelo gemini-2.0-flash-latest no existia en el endpoint v1beta. Consultando la lista de modelos disponibles con la API key se identifico gemini-flash-latest como el modelo correcto para esa cuenta.

5. Conclusiones y Aprendizajes

El desarrollo de GuardianClima ITBA nos permitio integrar de forma practica los conceptos trabajados en la materia:

- **Programacion:** aplicamos funciones, modulos, manejo de CSV y modularidad separando la logica en capas (main, service, lib).
- **Ciberseguridad:** implementamos validacion de contraseñas con criterios reales y entendimos la diferencia entre texto plano y hashing (bcrypt) en aplicaciones reales.
- **Cloud Computing y Conectividad:** consumimos dos APIs REST externas manejando errores de red, HTTP y respuestas inesperadas.
- **Inteligencia Artificial:** integramos Google Gemini para generar consejos en lenguaje natural a partir de parametros climaticos, aprendiendo a disenar prompts efectivos.
- **Analisis de Datos:** procesamos el historial en CSV para calcular estadisticas y generamos graficos en Excel con los datos recolectados.

Equipo Numero 1 — Lucas Garbate, Tomas Grinstein, Bautista Cavalitto, Facundo Gatti